

BAN 130: Python-Group Assignment 1

```
In [7]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

Part 1: Exploring and Filtering Data (10%)

```
In [49]: #Load the eu_occ.sas7bdat file and examine the column names and values.
```

```
In [50]: import pandas as pd
df = pd.read_sas('EU_OCC.sas7bdat')
df.shape
```

```
Out[50]: (4785, 6)
```

```
In [51]: # This section loads the EU_OCC SAS dataset into Python using pandas.
```

```
In [52]: print("Column Names:")
print(df.columns)
```

Column Names:

Index(['Geo', 'Country', 'YearMon', 'Hotel', 'ShortStay', 'Camp'], dtype='object')

```
In [53]: # The column names and sample values are reviewed to understand the structure of th
```

```
In [54]: #Filter the dataset to include only rows where YearMon starts with "2016".
```

```
In [55]: df['YearMon'] = df['YearMon'].astype(str)

Year_2016 = df[df['YearMon'].str.startswith('2016')]

print("Rows from 2016:")
print(Year_2016)
```

Rows from 2016:

	Geo	Country	YearMon	Hotel	ShortStay	Camp
9	b'AT'	b'Austria'	2016M12	6,670,483	1,468,847	117,579
10	b'AT'	b'Austria'	2016M11	3,600,616	681,867	28,303
11	b'AT'	b'Austria'	2016M10	5,727,389	985,402	146,108
12	b'AT'	b'Austria'	2016M09	7,726,801	1,443,829	620,032
13	b'AT'	b'Austria'	2016M08	11,399,594	3,022,261	1,897,979
...	...	...	...	...	...	...
4636	b'UK'	b'United Kingdom'	2016M05	14,811,451	7,820,198	6,237,354
4637	b'UK'	b'United Kingdom'	2016M04	13,788,240	5,619,281	4,665,842
4638	b'UK'	b'United Kingdom'	2016M03	12,239,798	4,663,424	1,524,859
4639	b'UK'	b'United Kingdom'	2016M02	10,478,239	3,022,689	337,125
4640	b'UK'	b'United Kingdom'	2016M01	9,789,043	2,612,609	497,162

[348 rows x 6 columns]

In [56]: *# The dataset is then filtered to keep only records where YearMon starts with 2016.*

In [57]: *#Format the columns Hotel, ShortStay, and Camp to include commas as thousand separa*

```
In [58]: print(df[['Hotel', 'ShortStay', 'Camp']].dtypes)

print(df[['Hotel', 'ShortStay', 'Camp']].head())

print(type(df['Hotel'].iloc[0]))
```

```
Hotel      float64
ShortStay  float64
Camp       float64
dtype: object
   Hotel  ShortStay   Camp
0  7,768,564  1,453,530  524,121
1 11,353,432  3,140,217 1,997,801
2 10,124,106  2,836,425 1,752,605
3  7,391,827  1,568,683  914,560
4  5,068,884  1,054,870  359,560
<class 'numpy.float64'>
```

```
In [59]: pd.options.display.float_format = '{:,.0f}'.format
print(df[['Hotel', 'ShortStay', 'Camp']].head())
```

```
   Hotel  ShortStay   Camp
0  7,768,564  1,453,530  524,121
1 11,353,432  3,140,217 1,997,801
2 10,124,106  2,836,425 1,752,605
3  7,391,827  1,568,683  914,560
4  5,068,884  1,054,870  359,560
```

```
In [60]: formatted_df = df.copy()
formatted_df['Hotel'] = formatted_df['Hotel'].map('{:,.0f}'.format)
formatted_df['ShortStay'] = formatted_df['ShortStay'].map('{:,.0f}'.format)
formatted_df['Camp'] = formatted_df['Camp'].map('{:,.0f}'.format)
print(formatted_df[['Hotel', 'ShortStay', 'Camp']])
```

	Hotel	ShortStay	Camp
0	7,768,564	1,453,530	524,121
1	11,353,432	3,140,217	1,997,801
2	10,124,106	2,836,425	1,752,605
3	7,391,827	1,568,683	914,560
4	5,068,884	1,054,870	359,560
...	...	...	...
4780	13,204,000	nan	nan
4781	13,252,000	nan	nan
4782	12,422,000	nan	nan
4783	10,271,000	nan	nan
4784	9,777,000	nan	nan

[4785 rows x 3 columns]

In [61]: *# Hotel, ShortStay, and Camp are formatted with comma separators to make large values*

In [62]: *#Drop the column Geo from the dataset.*

```
In [63]: df = df.drop(columns=['Geo'])
print(df.columns)
```

Index(['Country', 'YearMon', 'Hotel', 'ShortStay', 'Camp'], dtype='object')

In [64]: *# The Geo column is dropped from the output.*

Part 2: Creating and Formatting Columns (15%)

In [65]: *#Load the np_summary.sas7bdat file.*

```
In [66]: import pandas as pd
df = pd.read_sas('np_summary.sas7bdat')
df.shape
```

Out[66]: (135, 10)

In [67]: *# This section loads the np_summary SAS dataset.*

In [68]: *#Create a new column SqMiles by converting Acres to square miles (Acres * 0.0015625)*

```
In [69]: df['SqMiles'] = df['Acres'] * 0.0015625
print(df[['Acres', 'SqMiles']])
```

	Acres	SqMiles
0	649,096	1,014
1	669,650	1,046
2	1,750,716	2,735
3	2,523,512	3,943
4	2,697,391	4,215
..	...	...
130	30,351	47
131	28,243	44
132	36,347	57
133	138,306	216
134	574,453	898

[135 rows x 2 columns]

```
In [70]: # A new SqMiles column is created by converting Acres into square miles using the r
```

```
In [71]: print(df.columns)
```

```
Index(['Reg', 'Type', 'ParkName', 'DayVisits', 'OtherLodging', 'OtherCamping',
      'TentCampers', 'RVCampers', 'BackcountryCampers', 'Acres', 'SqMiles'],
      dtype='object')
```

```
In [72]: #Create a new column Camping as the sum of OtherCamping, TentCampers, RVCampers,
        #and BackcountryCampers.
```

```
In [74]: df['Camping'] = (df['OtherCamping'] + df['TentCampers'] + df['RVCampers'] + df['B
print(df[['Camping']])
```

	Camping
0	6,375
1	2,162
2	7,050
3	3,063
4	1,123
..	...
130	66,062
131	37,197
132	19,163
133	178,701
134	17,714

[135 rows x 1 columns]

```
In [75]: #Format SqMiles and Camping columns to include commas and zero decimal places.
```

```
In [29]: formatted_df = df.copy()
formatted_df['SqMiles'] = formatted_df['SqMiles'].map('{:,.0f}'.format)
formatted_df['Camping'] = formatted_df['Camping'].map('{:,.0f}'.format)

print(formatted_df[['SqMiles', 'Camping']])
```

	SqMiles	Camping
0	1,014	6,375
1	1,046	2,162
2	2,735	7,050
3	3,943	3,063
4	4,215	1,123
..	...	...
130	47	66,062
131	44	37,197
132	57	19,163
133	216	178,701
134	898	17,714

[135 rows x 2 columns]

```
In [ ]: # A new Camping column is created by adding OtherCamping, TentCampers, RVCampers, a
# SqMiles and Camping are formatted with commas and zero decimal places for cleaner
```

```
In [31]: #Save the updated dataset with only the following columns: Reg, ParkName, DayVisits
#OtherLodging, Acres, SqMiles, Camping.
```

```
In [32]: final_df = df[ ['Reg', 'ParkName', 'DayVisits', 'OtherLodging', 'Acres', 'SqMiles',
print(final_df)
```

	Reg	ParkName	DayVisits \
0	b'A'	b'Cape Krusenstern National Monument'	15,000
1	b'A'	b'Kenai Fjords National Park'	346,534
2	b'A'	b'Kobuk Valley National Park'	15,500
3	b'A'	b'Yukon-Charley Rivers National Preserve'	1,146
4	b'A'	b'Bering Land Bridge National Preserve'	2,642
..	...	...	...
130	b'SE'	b'Cape Hatteras National Seashore'	2,411,711
131	b'SE'	b'Cape Lookout National Seashore'	458,000
132	b'SE'	b'Cumberland Island National Seashore'	61,896
133	b'SE'	b'Gulf Islands National Seashore'	4,771,309
134	b'SE'	b'Big Cypress National Preserve'	1,102,148

	OtherLodging	Acres	SqMiles	Camping
0	0	649,096	1,014	6,375
1	0	669,650	1,046	2,162
2	0	1,750,716	2,735	7,050
3	0	2,523,512	3,943	3,063
4	0	2,697,391	4,215	1,123
..	...	...	...	...
130	0	30,351	47	66,062
131	10,118	28,243	44	37,197
132	0	36,347	57	19,163
133	0	138,306	216	178,701
134	0	574,453	898	17,714

[135 rows x 7 columns]

```
In [ ]: # The final dataset is limited to the required columns only:
#Reg, ParkName, DayVisits, OtherLodging, Acres, SqMiles, and Camping.
```

Part 3: Data Transformation and Reporting (15%)

```
In [33]: df = pd.read_sas('np_summary.sas7bdat')
df['Type'] = df['Type'].str.decode('utf-8') #Type column was stored in "bytes" thef

#Add a new column ParkType using conditional logic:
df['ParkType'] = 'Other'

df.loc[df['Type'] == 'NM', 'ParkType'] = 'Monument'
df.loc[df['Type'] == 'NP', 'ParkType'] = 'Park'
df.loc[df['Type'].isin(['NPRE', 'PRE', 'PRESERVE']), 'ParkType'] = 'Preserve'
df.loc[df['Type'] == 'NS', 'ParkType'] = 'Seashore'
df.loc[df['Type'].isin(['RVR', 'RIVERWAYS']), 'ParkType'] = 'River'

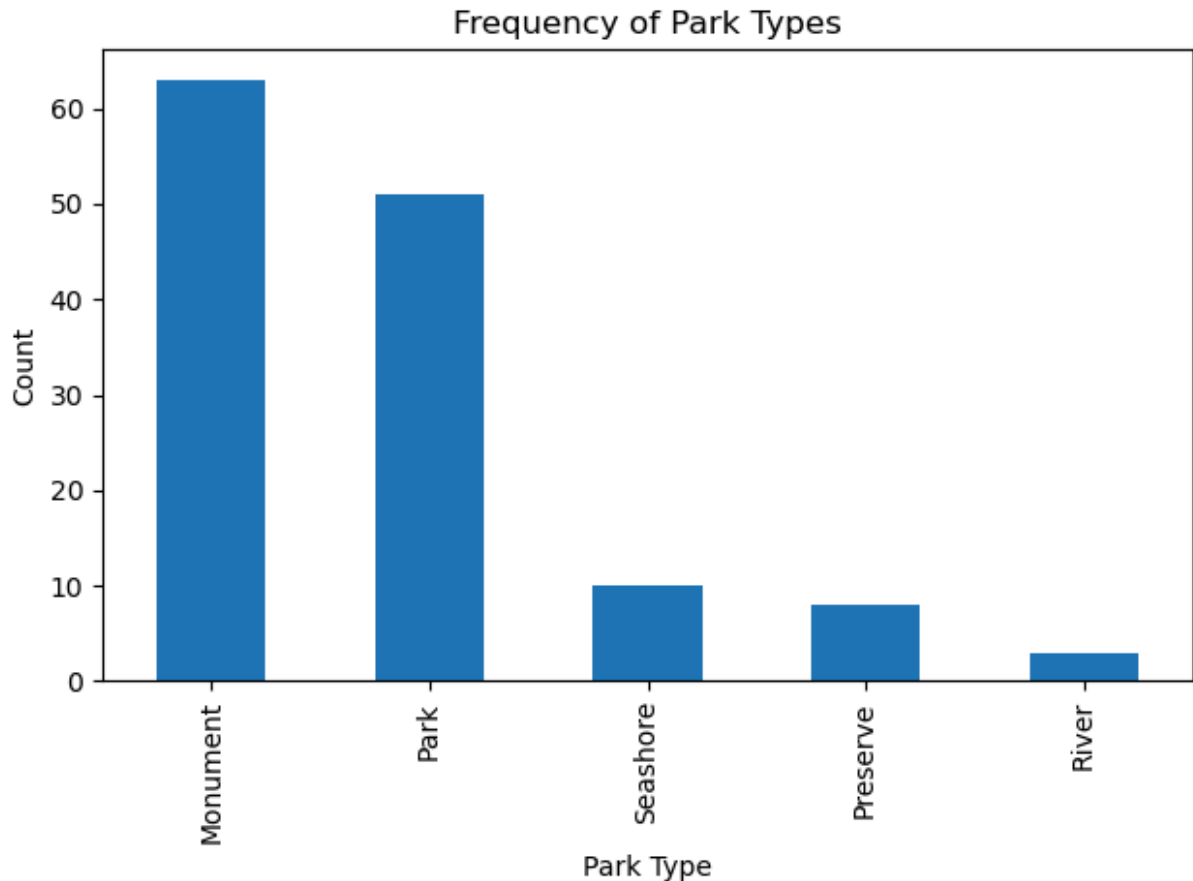
# Generate frequency report
freq_report = df["ParkType"].value_counts()

print(freq_report)

# Create bar chart
freq_report.plot(kind="bar")

plt.title("Frequency of Park Types")
plt.xlabel("Park Type")
plt.ylabel("Count")
plt.tight_layout()
plt.show()
```

```
ParkType
Monument    63
Park        51
Seashore    10
Preserve     8
River        3
Name: count, dtype: int64
```



```
In [ ]: # This section reloads the np_summary dataset and creates a new ParkType column.
# Conditional logic is used to convert park type codes into readable labels such as
# A frequency report is created to count how many parks fall into each ParkType cat
# A bar chart is generated to visually display the ParkType frequency results.
```

```
In [ ]:
```

Part 4: Advanced Queries and Data Formatting (30%)

```
In [34]: df = pd.read_sas("customer.sas7bdat")

query1 = df[(df["BankID"].isna()) & (df["CreditScore"] > 700)]
# Sort by Income descending
query1 = query1.sort_values(by="Income", ascending=False)

# Format Income in dollar format
query1["Income"] = query1["Income"].apply(lambda x: f"${x:,.2f}")

#nprint income column to check
print("Query 1 Results:")
print(query1[['Income']])
```

```

# Employed column was stored in bytes thefore I converted into str in order to be o
df["Employed"] = df["Employed"].str.decode('utf-8').str.strip()

# Convert SAS date format (days since 1960-01-01) to datetime
df['DOB'] = pd.to_datetime(df['DOB'], unit='d', origin='1960-01-01')

# Filter rows where DOB is earlier than 1940-12-31 and Employed is Y
query2 = df[(df['DOB'] < '1940-12-31') & (df['Employed'] == 'Y')]

# Sort by DOB descending
query2 = query2.sort_values(by='DOB', ascending=False)

# print DOB and Employed Column to check
print("Query 2 Results:")
print(query2[["DOB", "Employed"]])

```

Query 1 Results:

	Income
3093	\$102,435.14
1383	\$101,684.04
1942	\$96,699.91
1102	\$91,189.42
2034	\$88,942.84
...	...
4886	\$nan
4914	\$nan
4918	\$nan
4926	\$nan
4928	\$nan

[1291 rows x 1 columns]

Query 2 Results:

	DOB	Employed
19475	1940-10-16	Y
68001	1940-07-17	Y
16201	1940-07-14	Y
18817	1940-06-26	Y
56967	1940-06-09	Y
...	...	...
35988	1927-11-12	Y
79293	1919-04-30	Y
28930	1918-08-05	Y
29994	1917-09-26	Y
96361	1915-05-25	Y

[81 rows x 2 columns]

```

In [ ]: # This section loads the customer SAS dataset for customer-based filtering and repo
# The first query filters customers where BankID is missing and CreditScore is grea
# The results are sorted by Income from highest to lowest, and Income is formatted
# The second query filters customers with a DOB before 1940-12-31 and Employed equa
# These results are sorted by DOB in descending order to show the most recent dates

```


Part 5: Case-Based Transformation (30%)

```
In [44]: # Load the customer dataset
customer = pd.read_sas("customer.sas7bdat")
```

```
In [ ]: # This section uses the customer dataset to create new business-friendly categories
```

```
In [45]: # Create CreditCategory column
conditions = [
    customer["CreditScore"] >= 750,
    customer["CreditScore"].between(700, 749),
    customer["CreditScore"].between(650, 699),
    customer["CreditScore"].between(550, 649),
    customer["CreditScore"] < 550
]

choices = [
    "Excellent",
    "Good",
    "Fair",
    "Poor",
    "Bad"
]

customer["CreditCategory"] = np.select(
    conditions,
    choices,
    default="Unknown"
)
```

```
In [ ]: # A CreditCategory column is created by grouping customers based on their CreditScore
# Customers are classified as Excellent, Good, Fair, Poor, or Bad based on the requirements
```

```
In [46]: # Create MaritalStatus column using the existing Married column
customer["MaritalStatus"] = customer["Married"].map({
    b"M": "Married",
    b"D": "Divorced",
    b"S": "Single",
    b"W": "Widowed"
}).fillna("Unknown")
```

```
In [ ]: # A MaritalStatus column is created by converting marital status codes into readable text
```

```
In [47]: # Display results
print(customer[["CreditScore", "CreditCategory", "Married", "MaritalStatus"]].head(10))
```

	CreditScore	CreditCategory	Married	MaritalStatus
0	711	Good	b'M'	Married
1	750	Excellent	NaN	Unknown
2	790	Excellent	b'M'	Married
3	635	Poor	b'S'	Single
4	716	Good	b'M'	Married
5	684	Fair	b'S'	Single
6	617	Poor	b'M'	Married
7	639	Poor	b'M'	Married
8	687	Fair	b'M'	Married
9	NaN	Unknown	b'S'	Single

```
In [ ]: # A MaritalStatus column is created by converting marital status codes into readabl  
# Any marital status code outside M, D, S, or W is labelled as Unknown.
```